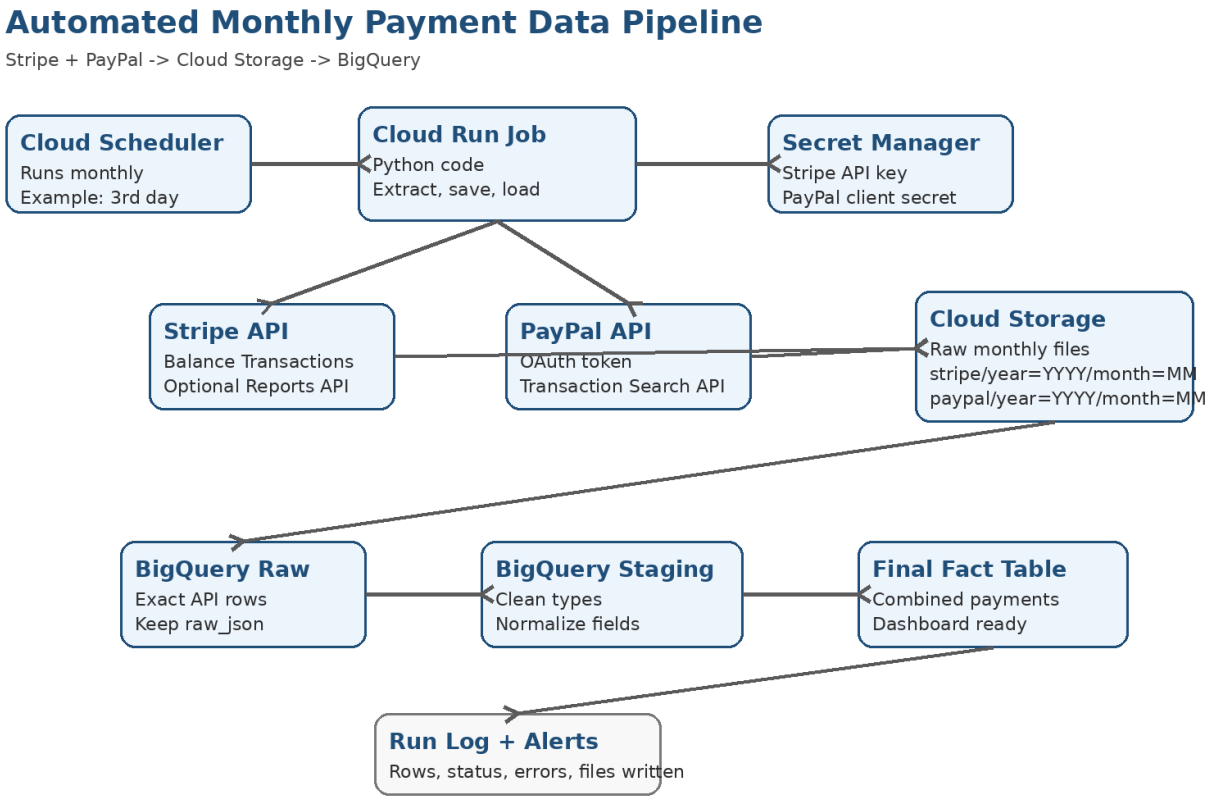
Stripe + PayPal to BigQuery

Automated Monthly Payment Data Pipeline Plan

Junior Developer View | Prepared for Ancient Remedies | June 11, 2026



One-line answer

Yes, the monthly extraction can be automated. Use Cloud Scheduler to start a Cloud Run Job, read API credentials from Secret Manager, extract Stripe and PayPal data, store raw files in Cloud Storage, load raw tables in BigQuery, then transform them into a final combined payments table.

Contents

- 1. Goal and recommended version 1
- 2. Full automated flow
- 3. Why these APIs
- 4. Google Cloud components
- 5. Monthly run logic
- 6. BigQuery table design
- 7. Developer build steps
- 8. Data quality, duplicates, and reconciliation
- 9. Security and permissions
- 10. Production runbook
- 11. Future improvements
- 12. Official documentation references

1. Goal and recommended version 1

Goal: replace manual monthly Stripe and PayPal downloads with an automated pipeline that lands the data in BigQuery every month.

Recommended version 1: use Stripe Balance Transactions API for Stripe, PayPal Transaction Search API for PayPal, Cloud Storage for raw backup, and BigQuery for raw, staging, and final reporting tables.

Optional later: add Stripe Reports API and PayPal SFTP/monthly statement files if the finance team needs exact platform-generated CSV reports.

Version 1 scope

- Pull previous full month automatically, for example May data on June 3.
- Store raw API output before transforming anything.
- Load raw data into BigQuery with ingestion metadata.
- Create clean staging tables with standardized amounts, currencies, dates, and IDs.
- Merge Stripe and PayPal into one final fact table for reporting.
- Write run logs and send success/failure alerts.

High-level decision

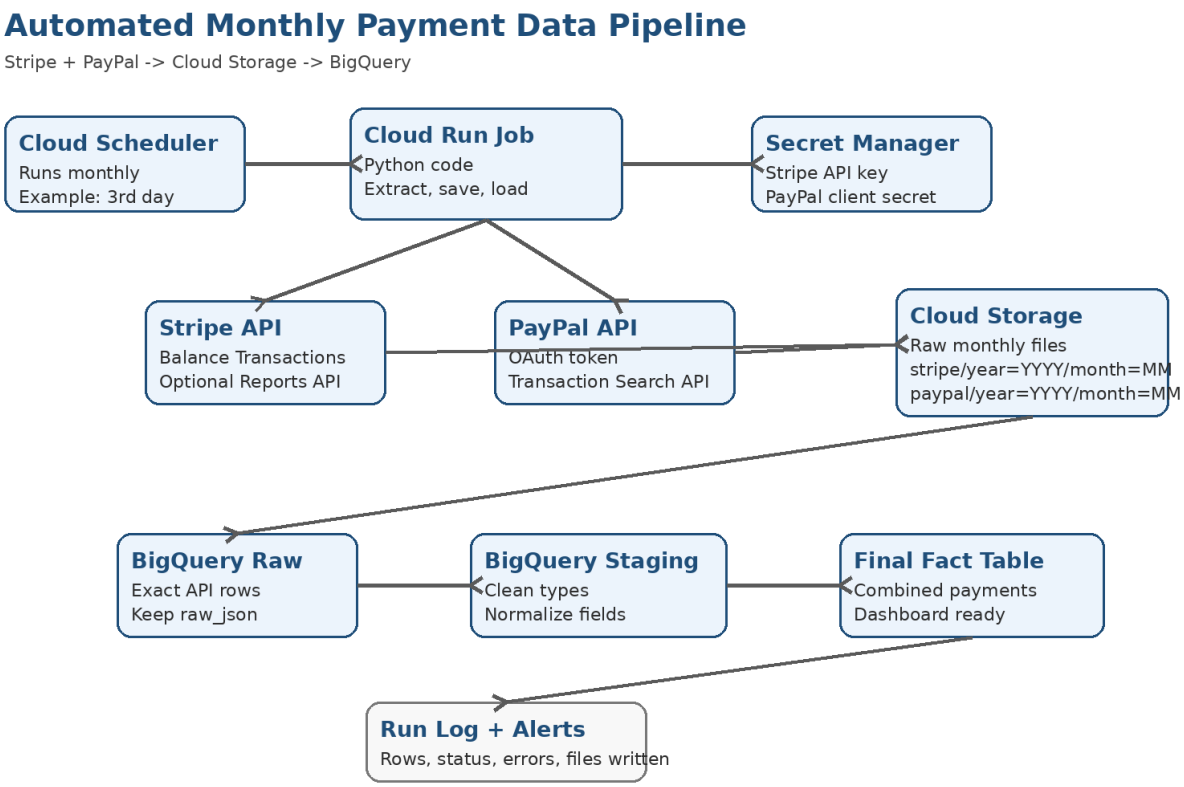
Area	Choice for version 1	Reason
Orchestration	Cloud Scheduler + Cloud Run Job	Simple monthly schedule and enough runtime for API pagination.
Secrets	Secret Manager	Keeps Stripe and PayPal credentials out of code.
Raw backup	Cloud Storage	Keeps source files for audit/reprocessing.
Warehouse	BigQuery	Main analytics/reporting destination.
Stripe source	Balance Transactions API	Transaction-level JSON rows, good for BigQuery.
PayPal source	Transaction Search API	Official transaction-level API with pagination.
Transforms	BigQuery SQL	Easy to audit and rerun monthly transformations.

2. Full automated flow

This is the complete flow the junior developer should build.

```
Cloud Scheduler
  -> Cloud Run Job
     -> Read secrets from Secret Manager
     -> Extract Stripe data
     -> Extract PayPal data
     -> Save raw files to Cloud Storage
     -> Load raw files to BigQuery raw tables
     -> Run BigQuery staging/final SQL
     -> Write etl_run_log
     -> Send alert
```

Pipeline architecture



Recommended monthly schedule

Run the job on the 3rd day of every month. This gives enough buffer after month-end for PayPal/Stripe data availability and avoids pulling an incomplete current month.

```
Example:
Job runs on: 2026-06-03
Extract period: 2026-05-01T00:00:00Z to 2026-06-01T00:00:00Z
Reporting month: 2026-05
```

3. Why these APIs

Stripe options

Stripe option	Output	Use it when	Version 1 decision
Balance Transactions API	JSON transaction objects	You want transaction-level rows in BigQuery.	Use this first.
Reports API	Generated CSV report files	Finance wants Dashboard-style monthly reports such as balance, payout reconciliation, fees, or tax.	Add later if needed.

Stripe Balance Transactions API returns transaction history objects with fields such as id, amount, fee, net, currency, created, type, reporting_category, and source. Stripe list APIs use cursor pagination with starting_after, and the list balance transactions endpoint supports created date filters and a limit of 1 to 100 records per request.

Stripe Reports API is asynchronous: create a report run, wait for processing, then download the generated file. Stripe financial reports from the Dashboard are also available via API and can be scheduled automatically.

Important amount difference: Stripe API amounts are in minor units, such as cents for USD, while Stripe CSV reports use major currency units, such as 10.00 USD.

Stripe version 1 API request shape

```
GET https://api.stripe.com/v1/balance_transactions
  ?created[gte]=<previous_month_start_unix>
  &created[lt]=<current_month_start_unix>
  &limit=100

Auth:
  Authorization: Bearer STRIPE_SECRET_KEY

Pagination:
  while has_more is true:
      next request includes starting_after=<last_balance_transaction_id>
```

PayPal options

PayPal option	Output	Use it when	Version 1 decision
Transaction Search API	JSON transaction details	You want monthly transaction-level rows.	Use this first.
Balances API	Balance by currency at a point in time	You want opening/closing balance validation.	Use as validation later.
Monthly statement / SFTP reports	PayPal-generated report files	Finance wants official statement files or large report downloads.	Optional later.

PayPal Transaction Search API endpoint is GET /v1/reporting/transactions. It requires OAuth 2.0 access token authentication. Use start_date, end_date, fields=all, page_size=500, page, and balance_affecting_records_only=Y.

PayPal notes that executed transactions can take up to three hours to appear, the API lists transactions for the previous three years, the maximum date range is 31 days, and transaction_id is not unique in the reporting system.

PayPal version 1 API request shape

```
Step 1: Get OAuth token
POST https://api-m.paypal.com/v1/oauth2/token
  grant_type=client_credentials
  Basic auth: PAYPAL_CLIENT_ID:PAYPAL_CLIENT_SECRET

Step 2: List transactions
GET https://api-m.paypal.com/v1/reporting/transactions
  ?start_date=2026-05-01T00:00:00Z
  &end_date=2026-05-31T23:59:59Z
  &fields=all
  &page_size=500
  &page=1
  &balance_affecting_records_only=Y

Pagination:
  loop page=1 to total_pages
```

4. Google Cloud components

Component	Purpose	Developer notes
Cloud Scheduler	Starts the pipeline automatically every month.	Cron example: 0 6 3 * * in Asia/Kolkata or UTC based on team preference.
Cloud Run Job	Runs Python extraction and loading code.	Better than a small function if extraction may run longer or handle many pages.

Secret Manager	Stores API credentials safely.	Create secrets for STRIPE_SECRET_KEY, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET.
Cloud Storage	Stores raw monthly extract files.	Use partitioned folder paths by source, year, and month.
BigQuery	Stores raw, staging, final, and log tables.	Use partitioning by transaction_month or loaded_at.
Cloud Logging / Alerting	Tracks failures and operational details.	Send Slack/email alert after each monthly run.

Suggested GCS paths

```
gs://payment-data-raw/stripe/year=2026/month=05/stripe_balance_transactions_202605.ndjson
gs://payment-data-raw/paypal/year=2026/month=05/paypal_transactions_202605.ndjson
gs://payment-data-raw/logs/year=2026/month=05/run_<run_id>.json
```

5. Monthly run logic

1. Calculate previous full month in UTC. Example: if the job runs on 2026-06-03, period_start=2026-05-01 and period_end=2026-06-01.
2. Generate run_id, for example pay_extract_202605_<uuid>.
3. Read credentials from Secret Manager.
4. Extract Stripe balance transactions using created[gte] and created[lt].
5. Extract PayPal transactions using start_date, end_date, fields=all, page_size=500, and page loop.
6. Write raw responses as newline-delimited JSON to Cloud Storage.
7. Load raw ndJSON files into BigQuery raw tables.
8. Run SQL transforms into staging tables.
9. MERGE staging data into final fact table to avoid duplicates.
10. Write run status to etl_run_log and send an alert.

Simplified Python pseudocode

```
def main():
    run_id = create_run_id()
    period_start, period_end = previous_full_month()

    secrets = read_secrets_from_secret_manager()

    stripe_rows = extract_stripe_balance_transactions(
        api_key=secrets.stripe_key,
        start=period_start,
        end=period_end
    )

    paypal_token = get_paypal_oauth_token(
        client_id=secrets.paypal_client_id,
        client_secret=secrets.paypal_client_secret
    )

    paypal_rows = extract_paypal_transactions(
        token=paypal_token,
        start=period_start,
        end=period_end
    )

    stripe_file = write_ndjson_to_gcs('stripe', run_id, stripe_rows)
    paypal_file = write_ndjson_to_gcs('paypal', run_id, paypal_rows)
```

```
    load_gcs_to_bigquery(stripe_file, 'raw_stripe_balance_transactions')
    load_gcs_to_bigquery(paypal_file, 'raw_paypal_transactions')

    run_bigquery_sql('stg_stripe_transactions.sql')
    run_bigquery_sql('stg_paypal_transactions.sql')
    run_bigquery_sql('merge_fact_payment_transactions.sql')

    write_run_log(run_id, status='SUCCESS')
    send_alert('Payment pipeline succeeded')
```

6. BigQuery table design

Create one BigQuery dataset dedicated to this pipeline. Suggested name: finance_payments.

Tables

Table	Type	Purpose
finance_payments.raw_stripe_balance_transactions	Raw	One row per Stripe balance transaction object. Keep raw_json.
finance_payments.raw_paypal_transactions	Raw	One row per PayPal transaction detail object. Keep raw_json.
finance_payments.stg_stripe_transactions	Staging	Clean Stripe fields and convert minor-unit amounts to decimal amounts.
finance_payments.stg_paypal_transactions	Staging	Clean PayPal nested JSON fields and standardize names.
finance_payments.fact_payment_transactions	Final	Combined Stripe + PayPal table for reports and dashboards.
finance_payments.etl_run_log	Log	One row per run/source with status, row counts, errors, and file paths.

Raw Stripe table suggested columns

Column	Type	Notes
run_id	STRING	Pipeline run ID.
source	STRING	Always stripe.
balance_transaction_id	STRING	Stripe txn_* ID.
created_at	TIMESTAMP	Converted from Stripe Unix created.
available_on	TIMESTAMP	Converted from Unix available_on.
currency	STRING	Lowercase currency from API.
amount_minor	INT64	Amount in minor units, e.g. cents.
fee_minor	INT64	Fee in minor units.
net_minor	INT64	Net in minor units.
type	STRING	charge, refund, payout, transfer, etc.
reporting_category	STRING	Stripe reporting category.
source_object_id	STRING	Charge/refund/payout object ID.
raw_json	JSON or STRING	Full original object.
loaded_at	TIMESTAMP	Ingestion timestamp.

Raw PayPal table suggested columns

Column	Type	Notes
run_id	STRING	Pipeline run ID.
source	STRING	Always paypal.
paypal_account_id	STRING	PayPal account ID.
transaction_id	STRING	Not globally unique in PayPal reporting.
transaction_event_code	STRING	PayPal event code.
transaction_initiation_date	TIMESTAMP	Transaction start time.
transaction_updated_date	TIMESTAMP	Last update time.
transaction_status	STRING	S, P, D, V, etc.
currency	STRING	Currency code.
gross_amount	NUMERIC	transaction_amount.value.
fee_amount	NUMERIC	fee_amount.value.

invoice_id	STRING	Invoice/order reference if present.
payer_email	STRING	Customer email if returned and allowed.
raw_json	JSON or STRING	Full original transaction_details object.
loaded_at	TIMESTAMP	Ingestion timestamp.

Final table suggested columns

Column	Type	Purpose
payment_platform	STRING	stripe or paypal.
transaction_key	STRING	Deduplication key created by the pipeline.
transaction_id	STRING	Original source transaction ID.
transaction_date	TIMESTAMP	Standard transaction timestamp.
transaction_month	DATE	First day of month for partitioning/reporting.
transaction_type	STRING	charge, refund, payout, fee, etc.
status	STRING	Source status standardized where possible.
currency	STRING	Uppercase or lowercase standard, choose one.
gross_amount	NUMERIC	Positive/negative amount in major units.
fee_amount	NUMERIC	Fees in major units.
net_amount	NUMERIC	Net amount in major units.
customer_email	STRING	Nullable.
order_id	STRING	Invoice/custom/source reference if available.
run_id	STRING	Which run loaded the row.
loaded_at	TIMESTAMP	Ingestion timestamp.

MERGE approach for reruns

The pipeline must be rerunnable. If May 2026 is loaded twice, the final table should not duplicate rows. Use MERGE instead of blind INSERT.

```
MERGE finance_payments.fact_payment_transactions T
USING finance_payments.stg_combined_payment_transactions S
ON T.transaction_key = S.transaction_key
WHEN MATCHED THEN
  UPDATE SET
    status = S.status,
    gross_amount = S.gross_amount,
    fee_amount = S.fee_amount,
    net_amount = S.net_amount,
    loaded_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN
  INSERT (...) VALUES (...);
```

7. Developer build steps

Phase 1 - Access and setup

Done	Task
☐	Create or confirm GCP project and BigQuery dataset.
☐	Create Cloud Storage bucket for raw payment extracts.
☐	Create Secret Manager secrets: STRIPE_SECRET_KEY, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET.
☐	Create service account for Cloud Run Job.
☐	Grant least required permissions: read secrets, write GCS, load BigQuery, write logs.
☐	Confirm Stripe live API access and PayPal live app credentials.

Phase 2 - Local proof of concept

Done	Task
☐	Write local script to calculate previous month dates.
☐	Pull one month of Stripe balance transactions.
☐	Pull one month of PayPal transactions using OAuth and pagination.
☐	Write ndJSON files locally.
☐	Manually load files into BigQuery test tables.
☐	Validate row counts and sample transactions against Stripe/PayPal dashboards.

Phase 3 - Cloud deployment

Done	Task
☐	Package Python script into Docker image.
☐	Deploy as Cloud Run Job.
☐	Test manual execution of the Cloud Run Job.
☐	Verify raw files are written to Cloud Storage.
☐	Verify BigQuery raw loads succeed.
☐	Verify staging/final SQL runs after raw loads.

Phase 4 - Monthly automation

Done	Task
☐	Create Cloud Scheduler trigger for the 3rd of every month.
☐	Connect Scheduler to Cloud Run Job.
☐	Add retries for temporary API/network failures.
☐	Add etl_run_log writes for success/failure.
☐	Add Slack or email alert.
☐	Do one dry run for a previous month and confirm no duplicates on rerun.

8. Data quality, duplicates, and reconciliation

Duplicate handling

Stripe: use balance_transaction_id as the core unique source ID. For final transaction_key, use stripe + balance_transaction_id.

PayPal: do not use transaction_id alone because PayPal says the same transaction ID can appear more than once in the reporting system. Use a compound key.

```
Stripe transaction_key:
  stripe || ':' || balance_transaction_id

PayPal transaction_key example:
  paypal || ':' || transaction_id || ':' || transaction_event_code || ':' || transaction_updated_date ||
':' || amount || ':' || currency
```

Quality checks after each run

Check	How to check	Fail action
API row count	Rows extracted vs rows loaded.	Fail run or mark PARTIAL_SUCCESS.
Duplicate count	Count duplicate transaction_key in staging/final.	Block merge until fixed.
Null critical IDs	transaction_id and dates should not be null.	Review raw response and schema.
Amount parsing	Stripe minor units converted; PayPal decimal strings parsed.	Fix transform SQL.
Month boundary	All rows belong to requested reporting	Review date filter logic.

	month or allowed late updates.	
Reconciliation	Compare total gross/fee/net by source to source dashboards/reports.	Investigate before publishing.

Run log example

```
run_id: pay_extract_202605_abc123
source: paypal
period_start: 2026-05-01T00:00:00Z
period_end: 2026-06-01T00:00:00Z
status: SUCCESS
rows_extracted: 3210
rows_loaded: 3210
gcs_path: gs://payment-data-raw/paypal/year=2026/month=05/paypal_transactions_202605.ndjson
started_at: 2026-06-03T00:30:00Z
finished_at: 2026-06-03T00:38:40Z
error_message: null
```

9. Security and permissions

- Never hardcode API keys in code or config files.
- Store Stripe and PayPal credentials in Secret Manager only.
- Use separate sandbox and live secrets.
- Give the Cloud Run service account only the permissions it needs.
- Restrict raw GCS bucket access because it may include customer emails, names, invoice IDs, and transaction details.
- Mask or limit PII columns in downstream dashboards if business users do not need them.
- Rotate API keys on a defined schedule and after team/member changes.

Minimum service account permissions

Permission area	Needed for
Secret Manager secret accessor	Read Stripe and PayPal credentials.
Storage object creator/admin on raw bucket	Write raw extract files and logs.
BigQuery job user	Run BigQuery load and SQL jobs.
BigQuery data editor on finance_payments dataset	Insert/merge raw, staging, final, and log tables.
Cloud Logging writer	Write logs from Cloud Run.

10. Production runbook

Normal monthly operation

11. Cloud Scheduler triggers the Cloud Run Job automatically.
12. Developer/finance receives success alert with source row counts.
13. Developer verifies etl_run_log status is SUCCESS for both Stripe and PayPal.
14. Finance or analytics validates final totals in dashboard/report.

Manual rerun process

```
Input parameters:
  source: stripe, paypal, or both
  period_start: YYYY-MM-01
  period_end: next month YYYY-MM-01
  mode: rerun

Expected behavior:
  1. Pull data again.
```

```
2. Write new raw files with new run_id.
3. Rebuild staging.
4. MERGE into final table.
5. No duplicate final rows.
```

Common failures and fixes

Failure	Likely cause	Fix
Stripe 401/403	Wrong/expired API key or live/test mismatch.	Check Secret Manager key and Stripe mode.
PayPal 401	OAuth credentials invalid or wrong environment.	Check client_id/client_secret and live vs sandbox base URL.
PayPal date error	Date range too large or invalid format.	Keep range <=31 days and use ISO timestamp with seconds.
BigQuery schema error	API returned new or nested fields.	Load raw_json as JSON/STRING and update staging SQL carefully.
Duplicate rows	Blind insert instead of MERGE.	Use transaction_key and MERGE logic.
Missing latest transactions	Ran too early after month-end.	Schedule after buffer period, e.g. 3rd day.

11. Future improvements

- Add Stripe Reports API for official monthly finance CSVs such as balance, payout reconciliation, fees, and tax reports.
- Add PayPal Balances API to store opening and closing balance snapshots by currency.
- Add PayPal SFTP report ingestion if PayPal enables report files for the account and finance wants official report files.
- Add daily incremental extraction in addition to monthly close, so data is fresher during the month.
- Add reconciliation dashboard in Looker Studio or another BI tool.
- Add automated anomaly checks, for example fee percentage spikes or sudden row count drops.
- Add Terraform for repeatable deployment of GCP resources.

12. Official documentation references

[1] Stripe - How the Reports API works: https://docs.stripe.com/reports/api

[2] Stripe - Report Runs API reference: https://docs.stripe.com/api/reporting/report_run

[3] Stripe - Balance Transactions API list: https://docs.stripe.com/api/balance_transactions/list

[4] Stripe - Pagination: https://docs.stripe.com/api/pagination

[5] PayPal - REST API authentication: https://developer.paypal.com/api/rest/authentication/

[6] PayPal - Transaction Search API: https://developer.paypal.com/docs/api/transaction-search/v1/

[7] Google Cloud - Cloud Scheduler documentation: https://docs.cloud.google.com/scheduler/docs

[8] Google Cloud - Execute Cloud Run jobs on a schedule: https://docs.cloud.google.com/run/docs/execute/jobs-on-schedule

[9] Google Cloud - Secret Manager documentation: https://docs.cloud.google.com/secret-manager/docs

[10] Google Cloud - Loading JSON data from Cloud Storage into BigQuery:
https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-json